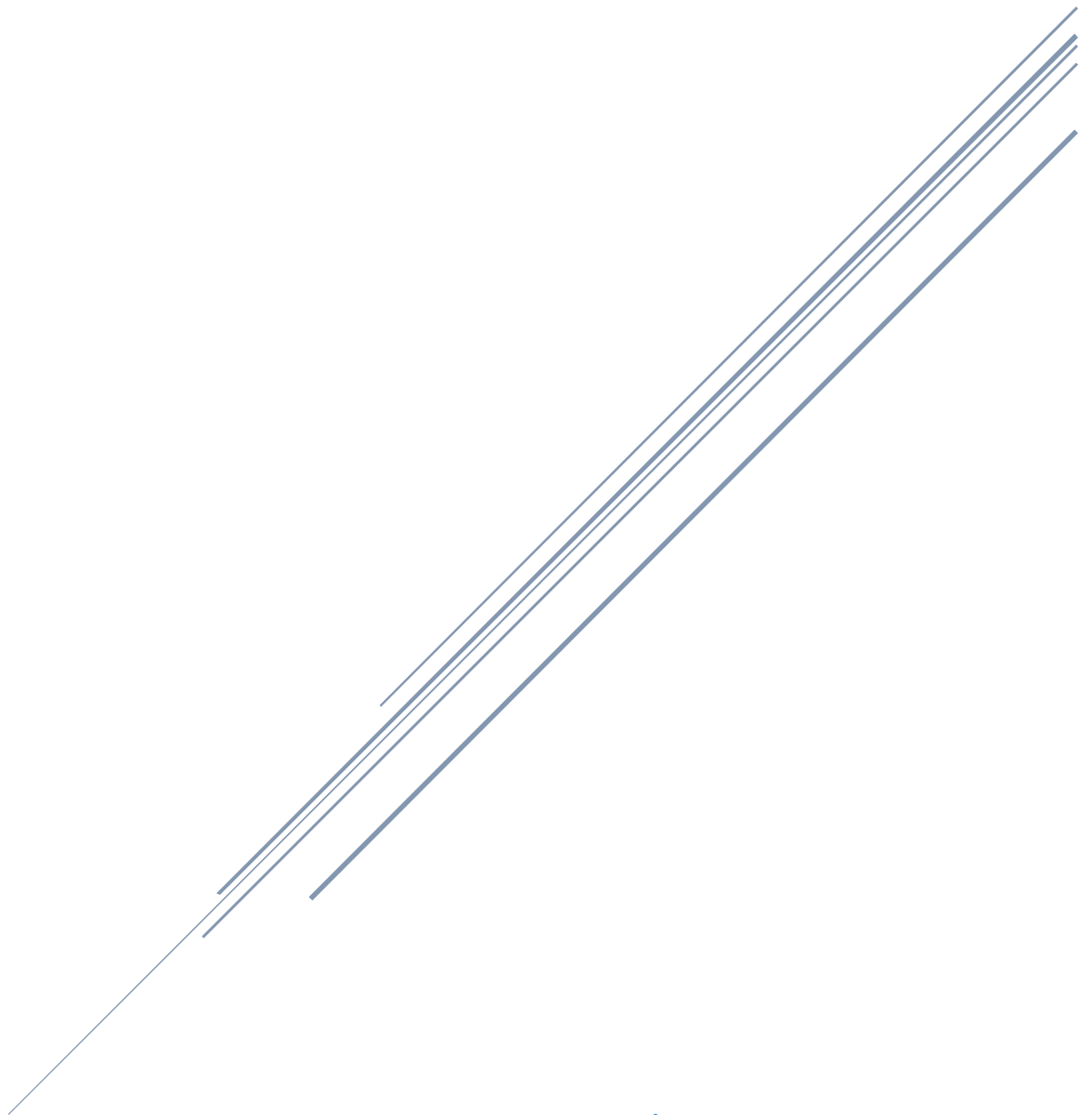


# CODE AND SUMMARY OF CLASS 13

Sahir Ahmed Sheikh

Saturday (2 – 5)



Teachers:

**Muhammad Bilal And Ali Aftab Sheikh**

## Code And Summary Of Class 13 – Saturday (2 – 5) | Quarter 3

### Introduction

This document provides a detailed summary of Class 13 of Quarter 3, held on Saturday from 2 to 5 PM, led by Sir Ali Aftab Sheikh and Sir Muhammad Bilal. The class focused on reinforcing the concepts of **AI agents**, particularly through the use of **Open Router**, **Light LLM**, and **UV** for managing Python projects. The session included practical coding demonstrations, discussions on **SWARM** and **Agent SDK**, and emphasized the importance of completing assignments to prepare for the upcoming test and Quarter 4.

The class addressed the following key objectives:

- Revisiting AI agent concepts with practical coding examples using Open Router and Light LLM.
- Introducing SWARM and its role in multi-agent systems.
- Explaining UV for managing Python virtual environments.
- Emphasizing the importance of assignment completion and preparation for the upcoming test.
- Preparing students for the transition to Quarter 4, which will introduce advanced topics.

Students were reminded of the limited classes remaining in Quarter 3, the strict policy on assignment completion, and the need to push code to GitHub. The session also included motivational advice to take the course seriously and prepare for practical, hands-on test questions.

## Topics Covered

### *1. Class Updates and Quarter 3 Progress*

- **Quarter 3 Overview:** Quarter 3 is nearing its end, with only a few classes remaining. The environment was described as manageable, but Quarter 4 was introduced as significantly more challenging, requiring immediate preparation.
- **Test Updates:** A test is scheduled within July, with practical questions requiring hands-on coding experience. Students were advised to re view documentation and run code to prepare effectively.
- **Assignment Policy:** A new policy was introduced to check for incomplete assignments rather than completed ones. Students with incomplete assignments risk elimination from the course unless they have a genuine reason.
- **CR and Team Roles:** Class Representatives (CRs) and CR assistants were tasked with teaching topics and checking assignments. Students were encouraged to complete assignments to qualify for roles like CR, CR assistant, or elite member in Quarter 4.
- **Motivational Advice:** Students were urged to avoid procrastination, practice coding regularly, and use tools like NotebookLM for efficient learning through summaries, audio, and interactive modes.

### *2. Revision Of AI Agent Concepts:*

The session revisited AI agent concepts, focusing on their autonomous nature and the role of system prompts in defining their behavior. Key points included:

- **Agents:** Autonomous decision-makers that perform tasks based on a single prompt, unlike chatbots that require continuous input.
- **System Prompts:** Define the agent's specialization (e.g., math expert), enabling precise responses. Without prompts, agents may give generic answers, but context improves performance over time.
- **Guardrails:** Used to restrict agents to relevant tasks, though not covered in detail in this class.

### 3. Modular AI Infrastructure: SWARM Framework, UV, Open Router, Light LLMs

#### 3.1 SWARM Framework

- **Definition:** SWARM is an experimental Open AI framework for lightweight, autonomous orchestration of multi-agent systems, focusing on agents and hands-off task assignment.
- **Real-World Analogy:** Compared to a cricket team where each player (agent) has a specific role (e.g., batsman, bowler), and a captain (orchestrator) assigns tasks.
- **Key Components:**
  - **Agents:** Specialized for specific tasks (e.g., decoration, cashier).
  - **Hands-Off:** Task delegation to the appropriate agent, similar to a manager assigning tasks in a company.
- **Agent SDK:** An evolved version of SWARM, incorporating Anthropic's design patterns like chaining, routing, parallelization, orchestrator workers, and evolutionary optimizer for enhanced agent functionality.

#### 3.2 UV (Universal Virtual Environment)

- **Definition:** UV is a high-performance Python package manager written in Rust, designed to manage virtual environments for Python projects.
- **Purpose:** Prevents conflicts between project dependencies by creating isolated virtual environments, ensuring each project uses its required package versions.
- **Real-World Analogy:** Separating project dependencies is like organizing tools in separate boxes to avoid mixing them up.
- **Features:**
  - Lightning-fast package installation due to parallel processing.
  - Automatic virtual environment creation.
  - Compatibility with existing Python tools and support for modern packaging standards.

- **Coding Example:**

```
# Initialize a UV project
uv init class13
cd class13
# Install packages
uv add openai agents python-dotenv
```

- Explanation: The commands initialize a UV project, create a virtual environment, and install required packages, which are listed in the *pyproject.toml* file.

### 3.3 Open Router

- **Definition:** Open Router is an online platform providing access to multiple LLMs through a single API key, simplifying configuration for models like DeepSeek, Mistral, and Gemini.
- **Real-World Analogy:** Compared to a SIM card (API key) accessing multiple numbers (LLMs).
- **Benefits:** Simplifies switching between models by changing only the model name, supports free and paid models, and uses a unified base URL (<https://openrouter.ai/api/v1>).
- **How Open Router Works:** Open Router acts as a middleman between your code and many AI models. Instead of creating separate APIs for each model (like DeepSeek, Mistral), you just connect to Open Router. You send a request with your API key and specify which model you want to use — and it handles the rest for you!
- **Why Use Open Router Instead of Direct APIs?**

If you use direct APIs for each model, you'll need to manage **different keys, URLs, and formats**. But Open Router gives you:

- One API key
- One base URL
- One format

This saves time and reduces errors in your code.

- **Coding Example:**

```
import os
import asyncio
from openai import AsyncOpenAI

from agents import Agent, Runner

# Load environment variables
from dotenv import load_dotenv
load_dotenv()

# Configure client
client = AsyncOpenAI(
    api_key=os.getenv("OPEN_ROUTER_API_KEY"),
    base_url="https://openrouter.ai/api/v1"
)

# Define model and agent
model = "deepseek:free"
agent = Agent(
    name="Assistant",
    instructions="You are an expert math teacher.",
    model=OpenAIChatCompletionModel(model=model, client=client)
)

# Async function to run the agent
async def main():
    runner = Runner(agent=agent)
    result = await runner.run(user_input="What is PIAIC?")
    print(result)

# Run the async function
if __name__ == "__main__":
    asyncio.run(main())
```

- **Explanation:** The code sets up an async client with Open Router's API key and base URL, defines a math expert agent, and runs an async function to process the query "What is PIAIC?". The output is: PIAIC stands for Presidential Initiative for Artificial Intelligence and Computing.

### 3.4 Light LLM

- **Definition:** Light LLM is a simplified framework for accessing LLMs, requiring only a model name and API key without additional configuration.
- **Benefits:** Easier to use than Open Router for free models, ideal for practice and connectivity.
- **Coding Example:**

```
import os
import asyncio
from agents.extensions.models.lightllm import LightLLM
from agents import Agent, Runner

# Load environment variables
from dotenv import load_dotenv
load_dotenv()

# Define model and agent
model = "gemini-2-5-pro"
agent = Agent(
    name="Assistant",
    instructions="You are an expert math teacher.",
    model=LightLLM(model, api_key=os.getenv("GEMINI_API_KEY"))
)

# Async function to run the agent
async def main():
    runner = Runner(agent=agent)
    result = await runner.run(user_input="What is PIAIC?")
    print(result)

# Run the async function
if __name__ == "__main__":
    asyncio.run(main())
```

- **Explanation:** The code uses Light LLM to create an agent with a Gemini API key, demonstrating a simpler setup compared to Open Router. The query What is PIAIC? is processed similarly.

---

#### 4. Practical Coding and Debugging

- **Environment Setup:** Students were guided to set up UV projects and manage .env files for secure API key storage.
- **Debugging Example:** Common errors, such as undefined API keys, were addressed by ensuring proper .env file setup and using `os.getenv` to load keys securely.
- **Validation Logic:** Emphasis was placed on not printing API keys to avoid exposure, reinforcing secure coding practices.

---

#### 5. Key Insights and Takeaways

- **Agents Power:** Agents automate repetitive tasks using LLMs as their brain, enhancing efficiency in workflows.
- **System Prompts:** Critical for defining agent behavior, improving response accuracy when tailored to specific tasks.
- **UVs Importance:** Simplifies project management by isolating dependencies, preventing conflicts, and speeding up package installation.
- **Open Router and Light LLM:** Facilitate easy access to multiple LLMs, with Open Router offering a unified API and Light LLM providing a simpler alternative for free models.
- **Test Preparation:** Practical coding experience is essential for the upcoming test, with resources like NotebookLM recommended for efficient learning.

---

#### 6. 📖 Class Resources and Assignments

- **Today's Class Code**

Students are encouraged to experiment with the provided examples (e.g., switching models in Open Router, running Light LLM) to deepen understanding.

- **Assignments**

- Build AI Agents**

- 1. Hello Agent:**

- Work on the Hello Agent project from the GitHub repository: [Panaversity](#).
    - Apply concepts like system prompts, async functions, and Open Router/Light LLM to build a functional agent.

- 2. Career Mentor Agent:**

- 💡 What It Does

- Guide students through career exploration using multi-agent support:

- Recommends career paths based on interests
      - Uses **Tool: `get_career_roadmap()`** to show skills needed for a chosen field
      - Hands off between agents:
        - ✓ **CareerAgent** (suggests fields),
        - ✓ **SkillAgent** (shows skill-building plans),
        - ✓ **JobAgent** (shares real-world job roles)

- ⚙ Uses:

- OpenAI Agent SDK + Runner
      - Tools: Skill Roadmap Generator
      - Handoff: Switch between Career, Skill, and Job Agents

- 🔔 *Tools and Handoff have not been covered in class yet. You are encouraged to explore these topics independently as part of this assignment. They will be explained in an upcoming session to support your understanding.*

- 3. AI Travel Designer Agent**

### ❓ What It Does

Plan a full travel experience by coordinating between specialized agents:

- Suggests destinations based on mood or interests
- Uses Tool: `get_flights()`, `suggest_hotels()` with mock data
- Hands off between:
  - ✓ `DestinationAgent` (finds places),
  - ✓ `BookingAgent` (simulates travel booking),
  - ✓ `ExploreAgent` (suggests attractions & food)

### ⚙️ Uses:

- OpenAI Agent SDK + Runner
- Tools: Travel Info Generator, Hotel Picker
- Handoff: Between Destination, Booking, and Explore Agents

🔔 *Tools and Handoff have not been covered in class yet. You are encouraged to explore these topics independently as part of this assignment. They will be explained in an upcoming session to support your understanding.*

## 4. Game Master Agent (Fantasy Adventure Game) 🎮

### ❓ What It Does

Run a text-based adventure game using multiple AI agents:

- Narrates an adventure story based on player choices
- Uses Tool: `roll_dice()` and `generate_event()` to control game flow
- Hands off between:
  - ✓ `NarratorAgent` (story progress),
  - ✓ `MonsterAgent` (combat phase),
  - ✓ `ItemAgent` (inventory & rewards)

### ⚙️ Uses:

- OpenAI Agent SDK + Runner
- Tools: Dice Roller, Event Generator
- Handoff: Dynamic switches between roles based on gameplay

 *Tools and Handoff have not been covered in class yet. You are encouraged to explore these topics independently as part of this assignment. They will be explained in an upcoming session to support your understanding*

### ✓ **Submission Checklist for All 3 Assignments**

*Requirements:*

- Use **at least 2 tools** and **2 agents** with **handoff** logic
- Must be implemented using OpenAI Agent SDK + Runner

 *What to Submit:*

- A well-structured **code folder**
  - A **README.md** explaining how your agent works and the flow between tools/agents
  -  **Deadline:** Before the next class
  -  **Submission:** Students were urged to overcome challenges (e.g., heat, complex concepts) and maintain consistent practice to build portfolios.
    - Upload your assignment with screenshots or a demo video on **LinkedIn**
    - Submit the link via this [Assignment Submission Form](#)
  - **Complete Previous Assignments:**  
Ensure the previous Student Agent assignment is completed and pushed to GitHub.
- 

### ***Important Reminder***

- Use documentation and tools like NotebookLM for test preparation, focusing on practical coding skills.

- The next class is critical, with assignment checks for incomplete submissions. All Agents projects must be on GitHub.
  - Stay active on Discord for class announcements and review the [Panaversity](#) repository for the syllabus.
- 

## **Final Words**

Your responsibility is to learn actively, participate confidently, and practice consistently. Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

## **Allah Hafiz – See you in the Next Class!**

Repo Link: [GIAIC-Sat-Afternoon](#)

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. You can go through each class code and its full detailed summary without any hesitation, and it will also help you prepare well for your quiz.

Thank You for Reading!

Hope you understood Class 13 well.

"Education is the key to unlock the golden door of freedom." – *George Washington Carver*